

Metody programowania

mgr inż. Katarzyna Dadek

2025

Laboratorium 7: Wzorce projektowe cz. 1

Wzorce projektowe to sprawdzone, wielokrotnie używane rozwiązania typowych problemów projektowych w programowaniu. Nie są gotowym kodem, ale **schematem myślenia**, który możesz zaadaptować do konkretnej sytuacji.

7.1 Wzorce projektowe

Notatka

Wzorce projektowe to uniwersalne, sprawdzone w praktyce rozwiązania często pojawiających się, powtarzalnych problemów projektowych. Pokazują powiązania i zależności pomiędzy klasami oraz obiektami i ułatwiają tworzenie, modyfikację oraz utrzymanie kodu źródłowego. Są **opisem rozwiązania**, a nie jego implementacją.

7.1.1 Czym są wzorce projektowe?

Stanowią ogólnie akceptowane **rozwiązania**, które programiści wypracowali w celu rozwiązania powszechnych problemów w projektowaniu oprogramowania. Jest to swoisty **zestaw reguł, technik i struktur**, które pomagają w tworzeniu oprogramowania o **wysokiej jakości**. Wzorce projektowe stanowią rodzaj **receptur** lub **szablonów** dla projektantów i programistów, pomagając im unikać błędów i problemów, które już zostały rozwiązane przez innych.



7.1.2 Dlaczego wzorce projektowe są ważne?

- **Poprawiają jakość oprogramowania** - pomagają tworzyć kod, który jest bardziej niezawodny, elastyczny i efektywny. Dzięki nim łatwiej uniknąć błędów i problemów w trakcie rozwoju oprogramowania.
- **Ułatwiają skalowanie i rozwijanie projektów** - pozwalają na bardziej płynne i łatwiejsze rozszerzanie oprogramowania w przyszłości. Dzięki nim projekt staje się bardziej przyjazny dla zmian i modyfikacji.
- **Przyspieszają proces projektowania i implementacji** - znając sprawdzone rozwiązania, programiści mogą oszczędzić czas na projektowaniu od podstaw i skoncentrować się na rozwiązywaniu unikalnych problemów projektu.
- **Pomagają w unikaniu błędów** - są oparte na doświadczeniach innych programistów, co oznacza, że pomagają unikać często spotykanych błędów i pułapek.

Notatka

Po co w ogóle używać wzorców projektowych?

- Szybsze i bardziej świadome projektowanie.
- Czytelniejszy, bardziej przewidywalny kod.
- Łatwiejsze utrzymanie i rozwój aplikacji.
- Unikanie typowych błędów architektonicznych.

7.1.3 Rodzaje wzorców

- **Kreacyjne (tworzenie obiektów)** - odpowiadają na pytanie: *jak tworzyć obiekty bez sztywnego powiązania z konkretnymi klasami?*
Przykłady: **Singleton**, **Factory Method**, **Builder**.
- **Strukturalne (budowa zależności)** - pomagają układać klasy i obiekty w większe struktury.
Przykłady: **Adapter**, **Decorator**, **Composite**.
- **Behawioralne (zachowania, komunikacja)** - ułatwiają zarządzanie przepływem danych i logiką między obiektami.
Przykłady: **Strategy**, **Observer**, **Command**.

Wskazówka

Kiedy faktycznie warto sięgać po wzorce projektowe?

- gdy widzisz powtarzający się problem projektowy,
- gdy zależy Ci na elastyczności i wymienności komponentów,
- gdy złożoność kodu zaczyna wymykać się spod kontroli,
- gdy kod chcesz łatwo testować i rozszerzać.

7.2 Kreacyjne wzorce projektowe

7.2.1 Singleton

Singleton to wzorec projektowy, którego celem jest **zapewnienie, że w programie istnieje tylko jedna instancja danej klasy**, a dostęp do niej jest globalnie kontrolowany. Stosuje się go wtedy, gdy wiele elementów systemu musi korzystać z tego samego, współdzielonego zasobu. Przykładami mogą być menedżer konfiguracji, logger, połączenie z bazą danych czy obiekt odpowiedzialny za komunikację.

Wzorec ten zapobiega tworzeniu wielu kopii danej klasy, wymuszając jednocześnie centralny punkt dostępu. Najczęściej realizuje się to poprzez prywatny konstruktor oraz statyczną metodę, która zwraca jedyną instancję obiektu. Dzięki temu obiekt jest tworzony tylko raz, a kolejne odwołania zwracają tę samą instancję.

Notatka

Singleton

- Jedna instancja klasy + globalny dostęp.
- Jeden punkt dostępu do obiektu.
- Często: leniwa inicjalizacja (tworzenie przy pierwszym użyciu).
- Zastosowanie: logger, konfiguracja, połączenia z bazą itp.
- Wymaga bezpiecznej inicjalizacji w środowisku wielowątkowym.
- Używać ostrożnie — może utrudniać testowanie i rozbudowę.
- Alternatywa: *Dependency Injection*.

Singleton bywa krytykowany, ponieważ **może prowadzić do ukrytych zależności i utrudniać testowanie**, szczególnie w dużych systemach. Z tego względu warto używać go oszczędnie i tylko tam, gdzie rzeczywiście istnieje potrzeba posiadania pojedynczej instancji.

Wskazówka

Przykładowa implementacja wzorca *Singleton* przedstawiona jest w dodatku A

7.2.2 Metoda wytwórcza

Metoda wytwórcza to wzorzec projektowy, którego celem jest **oddzielenie procesu tworzenia obiektów od ich wykorzystania**. Umożliwia definiowanie wspólnego interfejsu do tworzenia obiektów, pozostawiając decyzję o tym, jaki konkretny typ obiektu ma zostać utworzony, klasom pochodnym. Dzięki temu kod staje się bardziej elastyczny i łatwiej go rozszerzać bez modyfikowania już istniejących elementów.

Wzorzec ten jest używany wtedy, gdy klasa nie powinna zależeć od konkretnych klas produktów lub gdy chcemy umożliwić tworzenie różnych wariantów obiektów w zależności od potrzeb. Każda klasa pochodna implementuje własną wersję metody wytwórczej, która zwraca odpowiedni typ produktu.

Notatka

Metoda wytwórcza

- Oddziela tworzenie obiektów od konkretnych klas produktów.
- Wprowadza wspólną metodę tworzącą obiekty.
- Klasy pochodne dostarczają własne implementacje metody fabrycznej.
- Ułatwia rozszerzanie systemu.

Metoda wytwórcza pomaga redukować zależności, poprawia możliwość testowania kodu i sprzyja stosowaniu zasad SOLID¹, szczególnie zasady otwarte-zamknięte. Stosuje się ją m.in. w systemach, gdzie istnieje potrzeba tworzenia obiektów o wspólnym interfejsie, ale różniących się implementacją.

Wskazówka

Przykładowa implementacja wzorca *Metoda wytwórcza* przedstawiona jest w dodatku B

¹Zasady SOLID zostaną omówione w dalszej części kursu

7.3 Fabryka abstrakcyjna

Fabryka abstrakcyjna to wzorzec, który **dostarcza interfejs do tworzenia rodzin spójnych ze sobą obiektów**, bez ujawniania ich konkretnych klas. Umożliwia tworzenie całych zestawów produktów, które mają ze sobą współpracować (np. zestaw GUI Windows: okno, przycisk, checkbox; zestaw GUI macOS: okno, przycisk, checkbox).

Notatka

Fabryka abstrakcyjna:

- **Główna idea:** zamiast tworzyć pojedynczy obiekt, system otrzymuje fabrykę, która potrafi dostarczyć całą grupę powiązanych produktów.
- **Użycie:** gdy program ma wspierać różne warianty produktów i trzeba łatwo przełączać się między rodzinami (np. motywami, systemami, konfiguracjami sprzętowymi).
- **Zalety:** spójność produktów, możliwość łatwego przełączania zestawów, ukrycie szczegółów tworzenia obiektów.
- **Wady:** większa złożoność struktury, konieczność rozbudowy wielu klas przy wprowadzaniu nowych wariantów produktów.

Fabryka abstrakcyjna skupia się na tworzeniu całych rodzin powiązanych ze sobą obiektów. Gdy aplikacja potrzebuje kilku współpracujących elementów, które muszą pochodzić z jednego zestawu, fabryka abstrakcyjna **dostarcza wspólny interfejs pozwalający na ich jednoczesne tworzenie**. Dzięki temu wszystkie produkty są ze sobą spójne i kompatybilne. Ten wzorzec przydaje się szczególnie wtedy, gdy system ma obsługiwać różne warianty środowisk lub konfiguracji, a przełączanie między nimi powinno być możliwe bez modyfikowania logiki aplikacji.

Metoda wytwórcza działa na zdecydowanie mniejszą skalę. **Jej celem jest delegowanie tworzenia jednego konkretnego produktu do podklas poprzez nadpisywanie metody odpowiedzialnej za konstrukcję obiektu**. Dzięki temu kod korzystający z produktu nie musi znać jego nazwy klasy ani konkretnej implementacji. To rozwiązanie jest przydatne, gdy zależy nam na elastycznym rozszerzaniu systemu o nowe typy pojedynczych obiektów, bez kopiowania logiki tworzenia i bez usztywniania kodu.

Najważniejsza różnica między tymi wzorcami polega na zakresie działania. **Fabryka abstrakcyjna operuje na zestawach produktów i odpowiada za spójność całych rodzin, natomiast metoda wytwórcza koncentruje się na pojedynczym obiekcie i na tym, aby sposób jego tworzenia można było zmieniać poprzez dziedziczenie.** W efekcie fabryka abstrakcyjna jest rozwiązaniem bardziej rozbudowanym i stosowanym wtedy, gdy w grę wchodzi złożona architektura i wiele współpracujących elementów, podczas gdy metoda wytwórcza jest lżejsza i bardziej elastyczna tam, gdzie potrzebna jest wymiennność pojedynczych obiektów.

Wskazówka

Przykładowa implementacja wzorca *Fabryka abstrakcyjna* przedstawiona jest w dodatku C

7.4 Budowniczy

Budowniczy to wzorzec kreacyjny, który **rozdziela proces konstruowania złożonego obiektu od jego reprezentacji**. Dzięki temu ten sam proces może tworzyć różne warianty obiektów, a kod staje się bardziej elastyczny i czytelny. Zamiast wywoływać konstruktory z długą listą parametrów lub tworzyć trudne do zarządzania konfiguracje, budowniczy pozwala tworzyć obiekty krok po kroku. Klient korzysta z obiektu kierownika lub samodzielnie wywołuje kolejne metody budowania, otrzymując na końcu gotowy, kompletny produkt.

Notatka

Budowniczy

- **Rozdziela** tworzenie obiektu od jego struktury.
- Umożliwia **budowę krok po kroku**.
- Pozwala tworzyć **różne warianty** tego samego produktu.
- Redukuje potrzebę używania konstruktorów z wieloma parametrami.
- Sprawdza się przy **złożonych obiektach** z wieloma opcjonalnymi elementami.

Wskazówka

Przykładowa implementacja wzorca *Budowniczy* przedstawiona jest w dodatku D

7.5 Prototyp

Prototyp to wzorec kreacyjny, który **polega na tworzeniu nowych obiektów poprzez kopiowanie istniejących egzemplarzy – prototypów**. Zamiast konstruować obiekt od zera, program duplikuje go, co pozwala szybko tworzyć nowe instancje o podobnej konfiguracji. Jest to szczególnie przydatne w sytuacjach, gdy tworzenie obiektu jest kosztowne, skomplikowane albo wymaga wielu kroków inicjalizacji. Dzięki kopiowaniu zyskujemy większą elastyczność, a także możliwość łatwego tworzenia wariantów obiektów na podstawie już istniejących.

Notatka

Prototyp:

- Tworzenie obiektów odbywa się przez **klonowanie**.
- Przydatny, gdy **koszt tworzenia obiektu** jest wysoki.
- Pozwala tworzyć **warianty** na podstawie istniejących egzemplarzy.
- Daje możliwość kopiowania **złożonych konfiguracji**.
- Każdy prototyp implementuje własną metodę **clone**.

Wskazówka

Przykładowa implementacja wzorca *Prototyp* przedstawiona jest w dodatku E

7.6 Sprawozdanie 7

Rozszerz swoją obecną aplikację o mechanizmy oparte na wybranych wzorcach kreacyjnych, tak aby zwiększyć elastyczność, czytelność i możliwość dalszej rozbudowy projektu. Zidentyfikuj miejsca, w których proces tworzenia obiektów jest złożony, powtarzalny lub silnie powiązany z konkretnymi klasami, a następnie zastosuj odpowiedni wzorec kreacyjny (np. Budowniczego, Metodę wytwórczą, Fabrykę abstrakcyjną lub Prototyp).

Wybierz minimum jeden rzeczywisty fragment swojej aplikacji, w którym tworzenie obiektów powoduje problemy architektoniczne (np. wieloparametrowe konstruktory, trudne do testowania instancjonowanie, rozbudowane konfiguracje, powtarzające się fragmenty kodu). Przeprojektuj go, wprowadzając wybrany wzorec tak, aby powstawanie obiektów było bardziej kontrolowane, spójne i elastyczne.

Na koniec przygotuj podsumowanie dokonanych zmian, wskazując, jakie problemy

projektowe udało się rozwiązać dzięki zastosowaniu danego wzorca oraz jakie korzyści przynosi to w kontekście dalszego rozwoju i utrzymania aplikacji.

A Singleton

```
1  class Singleton {
2      private static instance: Singleton | null = null;
3
4      private constructor() {}
5
6      public static getInstance(): Singleton {
7          if (this.instance === null) {
8              this.instance = new Singleton();
9          }
10         return this.instance;
11     }
12
13     public doSomething(): void {
14         console.log("Execute.");
15     }
16 }
17
18 const s1 = Singleton.getInstance();
19 const s2 = Singleton.getInstance();
20
21 console.log(s1 === s2); // true
```

B Metoda wytwórcza

```
1  interface Product {
2      use(): void;
3  }
4
5  class ProductA implements Product {
6      use(): void { console.log("Use product A"); }
7  }
8
9  class ProductB implements Product {
10     use(): void { console.log("Use product B"); }
11 }
12
13 abstract class Creator {
14     abstract createProduct(): Product;
15     operation(): void {
16         const product = this.createProduct();
17         product.use();
18     }
19 }
20
21 class CreatorA extends Creator {
22     createProduct(): Product { return new ProductA(); }
23 }
24
25 class CreatorB extends Creator {
26     createProduct(): Product { return new ProductB(); }
27 }
28
29 const creatorA = new CreatorA();
30 creatorA.operation(); // Use product A
31
32 const creatorB = new CreatorB();
33 creatorB.operation(); // Use product B
```

C Fabryka abstrakcyjna

```
1 interface Button {
2     click(): void;
3 }
4 class WinButton implements Button {
5     click(): void {
6         console.log("Windows button");
7     }
8 }
9 class MacButton implements Button {
10    click(): void {
11        console.log("macOS button");
12    }
13 }
14
15 interface GUIFactory {
16    createButton(): Button;
17 }
18 class WinFactory implements GUIFactory {
19    createButton(): Button {
20        return new WinButton();
21    }
22 }
23 class MacFactory implements GUIFactory {
24    createButton(): Button {
25        return new MacButton();
26    }
27 }
28
29 class Application {
30    private button: Button;
31
32    constructor(factory: GUIFactory) {
33        this.button = factory.createButton();
34    }
35    run() {
36        this.button.click();
37    }
38 }
39
40 const factory: GUIFactory = new WinFactory();
41 const app = new Application(factory);
42 app.run();
```

D Budowniczy

```
1  class House {
2      walls: string;
3      roof: string;
4      garage: boolean;
5
6      constructor(walls: string, roof: string, garage: boolean) {
7          this.walls = walls;
8          this.roof = roof;
9          this.garage = garage;
10     }
11 }
12
13 class HouseBuilder {
14     private walls: string = "";
15     private roof: string = "";
16     private garage: boolean = false;
17
18     setWalls(walls: string): HouseBuilder {
19         this.walls = walls;
20         return this;
21     }
22     setRoof(roof: string): HouseBuilder {
23         this.roof = roof;
24         return this;
25     }
26     setGarage(garage: boolean): HouseBuilder {
27         this.garage = garage;
28         return this;
29     }
30     build(): House {
31         return new House(this.walls, this.roof, this.garage);
32     }
33 }
34
35 const house = new HouseBuilder()
36     .setWalls("brick")
37     .setRoof("basic")
38     .setGarage(true)
39     .build();
40
41 console.log(house);
```

E Prototyp

```
1 interface Prototype<T> {
2     clone(): T;
3 }
4
5 class Document implements Prototype<Document> {
6     title: string;
7     content: string;
8     metadata: Record<string, any>;
9
10    constructor(title: string, content: string, metadata: Record<
11        string, any>) {
12        this.title = title;
13        this.content = content;
14        this.metadata = metadata;
15    }
16
17    clone(): Document {
18        return new Document(
19            this.title,
20            this.content,
21            { ...this.metadata }
22        );
23    }
24
25    const original = new Document(
26        "Report",
27        "Content",
28        { author: "Jan", version: 1 }
29    );
30
31    const copy = original.clone();
32    copy.metadata.version = 2;
33
34    console.log(original.metadata.version); // 1
35    console.log(copy.metadata.version);    // 2
```